Лабораторная работа №8

Паттерны проектирования в Python

1. Цель работы

Научиться выбирать и применять порождающие и поведенческие паттерны проектирования.

2. Постановка индивидуального задания

Вариант 8. Фитнес-центр. Factory создаёт разовые, месячные и годовые абонементы. Strategy реализует стандартную цену, скидку и семейный тариф. Дополнительный паттерн — Observer. Также применяется Repository для отделения хранения от бизнес-логики.

3. Краткие теоретические сведения

Factory централизует создание объектов, Strategy инкапсулирует взаимозаменяемые алгоритмы, Observer уведомляет подписчиков о событиях, Repository изолирует хранение. Совместное применение уменьшает связанность компонентов и упрощает расширение программы.

4. Исходный код программы

Файл factory.py

```
from models import Membership
class MembershipFactory:
    PRICES={"разовый":500.0,"месячный":2500.0,"годовой":24000.0}
    @classmethod
    def create(cls,membership_id,client_name,kind,months=1):
        if kind not in cls.PRICES: raise ValueError("Неизвестный тип абонемента.")
        if months <= 0: raise ValueError("Количество месяцев должно быть положительным.")
        return Membership(membership_id,client_name,kind,months,cls.PRICES[kind])
```

Файл main.py

```
from repository import MembershipRepository
from observers import ConsoleObserver,HistoryObserver
from strategies import StandardStrategy,DiscountStrategy,FamilyStrategy
from service import MembershipService
```

```
def main():
    history=HistoryObserver();
service=MembershipService(MembershipRepository(),StandardStrategy(),[ConsoleObserver(),history])
    while True:
        print("\n=== Абонементы фитнес-центра ===")
        print("1. Создать абонемент  2. Показать все  3. Удалить абонемент")
        print("4. Выбрать стратегию цены  5. История событий  0. Выход")
        choice=input("Выберите пункт: ").strip()
        try:
            if choice=="1": print(service.create(input("Клиент: "),input("Тип (разовый/месячный/годовой):
").strip(),int(input("Месяцев: "))))
            elif choice=="2":
                for item in service.repository.get_all(): print(item)
            elif choice=="3": print(service.cancel(int(input("ID абонемента: "))))
            elif choice=="4":
                print("1. Стандартная 2. Скидка 3. Семейная")
                service.set_strategy({"1":StandardStrategy(),"2":DiscountStrategy(),"3":FamilyStrategy()}[input("Стратегия:
").strip()]); print("Стратегия изменена.")
            elif choice=="5": print(history.events)
            elif choice=="0": break
            else: print("Неизвестный пункт меню.")
        except (ValueError,KeyError) as exc: print("Ошибка:",exc)
if __name__=="__main__": main()
```

Файл models.py

```
from dataclasses import dataclass
from enum import Enum
class MembershipStatus(Enum): ACTIVE="активен"; EXPIRED="истёк"; FROZEN="заморожен"; CANCELLED="отменён"
@dataclass
class Membership:
    membership_id:int
    client_name:str
    kind:str
    months:int
    price:float
    status:MembershipStatus=MembershipStatus.ACTIVE
```

Файл observers.py

```
class MembershipObserver:
    def update(self,event,membership): raise NotImplementedError
class ConsoleObserver(MembershipObserver):
    def update(self,event,membership): print(f"[СОБЫТИЕ] {event}: абонемент №{membership.membership_id},
{membership.client_name}")
class HistoryObserver(MembershipObserver):
    def __init__(self): self.events=[]
    def update(self,event,membership):
self.events.append({"event":event,"id":membership.membership_id,"client":membership.client_name})
```

Файл repository.py

```
class MembershipRepository:
    def __init__(self): self._items={}
    def add(self,membership): self._items[membership.membership_id]=membership
    def get(self,membership_id): return self._items.get(membership_id)
    def get_all(self): return list(self._items.values())
    def delete(self,membership_id): return self._items.pop(membership_id,None)
```

Файл service.py

```
from factory import MembershipFactory
from repository import MembershipRepository
class MembershipService:
  def __init__(self,repository,strategy,observers=None):
    self.repository=repository; self.strategy=strategy; self.observers=list(observers or []); self._next_id=1
  def _notify(self,event,membership):
    for observer in self.observers: observer.update(event,membership)
  def create(self,client_name,kind,months=1):
    if not client_name.strip(): raise ValueError("Имя клиента не должно быть пустым.")
    membership=MembershipFactory.create(self._next_id,client_name.strip(),kind,months); self._next_id+=1
    membership.price=round(self.strategy.calculate(membership),2)
    self.repository.add(membership); self._notify("создан",membership); return membership
  def cancel(self,membership_id):
    membership=self.repository.get(membership_id)
    if membership is None: raise KeyError("Абонемент не найден.")
    self.repository.delete(membership_id); self._notify("удалён",membership); return membership
  def set_strategy(self,strategy): self.strategy=strategy
```

Файл strategies.py

```
class StandardStrategy:
  def calculate(self, membership): return membership.price
class DiscountStrategy:
  def calculate(self, membership): return membership.price * 0.9
class FamilyStrategy:
  def calculate(self, membership): return membership.price * 0.8
```

5. Объяснение работы программы

MembershipFactory создаёт абонементы заданного типа. Стратегии вычисляют базовую стоимость, стоимость со скидкой 10% или семейную цену со скидкой 20%. MembershipRepository хранит объекты, а ConsoleObserver и HistoryObserver получают события создания и удаления.

6. Примеры работы программы в виде снимков экрана

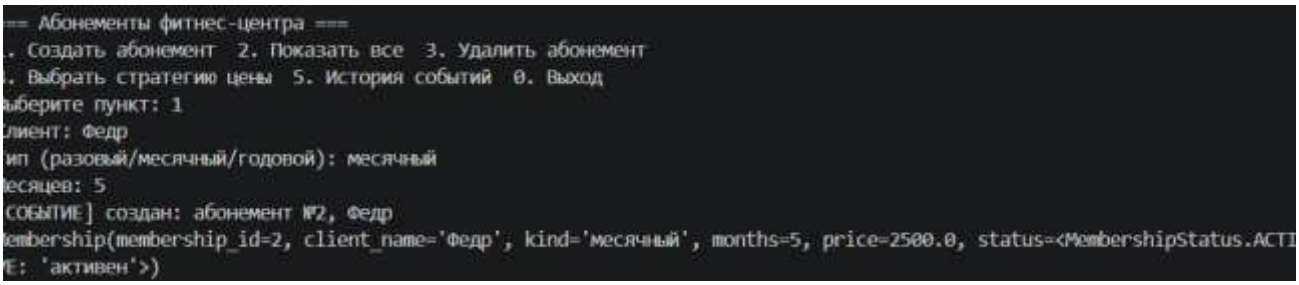


Рисунок 8.1 – Создание абонемента через Factory.

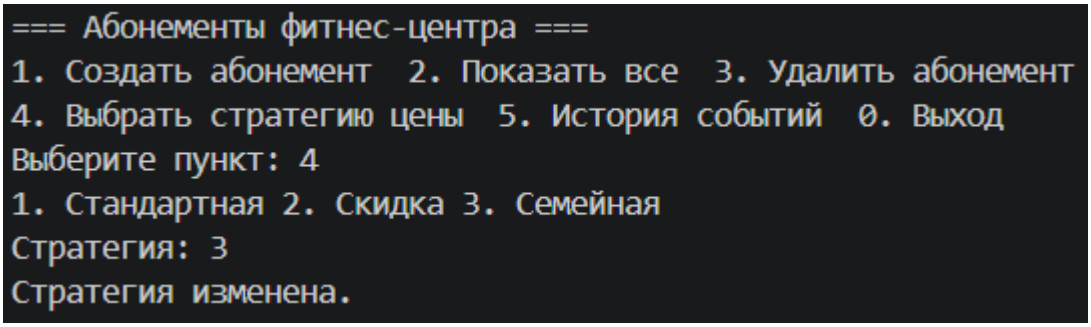


Рисунок 8.2 – Выбор стандартной, скидочной или семейной стратегии.

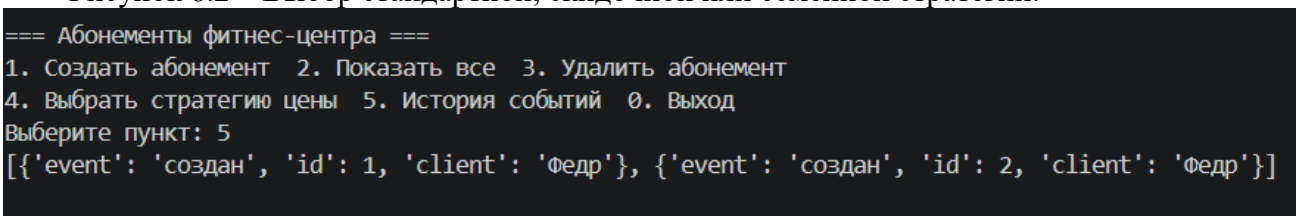


Рисунок 8.3 – Уведомления Observer и история зарегистрированных событий.

7. Вывод

В ходе выполнения лабораторной работы были реализованы требования индивидуального задания варианта 8. MembershipFactory создаёт абонементы заданного типа. Стратегии вычисляют базовую стоимость, стоимость со скидкой 10% или семейную цену со скидкой 20%. MembershipRepository хранит объекты, а ConsoleObserver и HistoryObserver получают события создания и удаления. Программа разделена на логические компоненты, содержит проверку типичных ошибок и сопровождается автоматическими тестами.

Выполнил: Куршаков Егор Игоревич